

Assignment 4 — Fingerprint Quality Assessment & Scoring Pipeline

YellowSense Technologies — Contactless Fingerprint Biometrics Assessment (FP-03)

1. Summary

This submission implements a five-metric image quality gate for contactless fingerprint captures (blur, brightness, glare, ROI completeness, ridge clarity), combines the metrics into a single 0-100 composite score, and exposes both a scripted batch test and a Streamlit UI with adjustable thresholds. The pipeline was validated on 20 self-generated placeholder captures (5 good, 5 blurry, 5 dark, 5 glare) and correctly identified the induced defect in 20/20 cases.

2. Methodology notes and design decisions

ROI segmentation: an initial implementation using global Otsu thresholding on raw pixel intensity mis-segmented captures whenever the ridge pattern's brightness range overlapped the background's average brightness. This was replaced with block-wise local-variance segmentation: ridge texture reliably produces far higher local variance than a background, regardless of absolute brightness, which is the same principle NIST-style block-variance segmentation uses for contact-scanner fingerprints.

Pass/fail gating: the composite score is a weighted average, which means one bad metric (e.g. a dark image) can be diluted by four good ones and still clear the 60-point cutoff. Since a single hard defect (blur, over/under-exposure, glare, incomplete ROI, or unclear ridges) is enough to break downstream minutiae extraction, *passed* requires both composite ≥ 60 AND no individual metric flag firing. The composite score is reported as a continuous "how good" signal on top of that veto, not a replacement for it.

Performance: the ridge-clarity check (an 8-orientation Gabor filter bank) was the dominant cost at full resolution (~830ms at 600x600 on CPU), well over the 150ms budget. Running the filter bank on a 256px-max working copy — ridge periodicity is a low/mid spatial-frequency pattern and survives downsampling — brought total `quality_gate()` latency to ~310-330ms on this reference CPU implementation, close to, but still slightly above, the <300ms target. A production Android build would close this gap further with a smaller kernel bank, a native (NDK/RenderScript) filter implementation, or GPU delegation.

3. Test data

Real phone captures were not available in this environment, so 20 synthetic placeholder images were generated (`generate_sample_images.py`) simulating ridge-like texture composited over a background, with degradations (Gaussian blur, exposure scaling, specular overlay) applied per category. These are clearly marked as placeholders in the code; before final submission they should be replaced with the 20 real phone photos required by Part D of the assignment brief. The pipeline logic and thresholds are independent of which images are used.

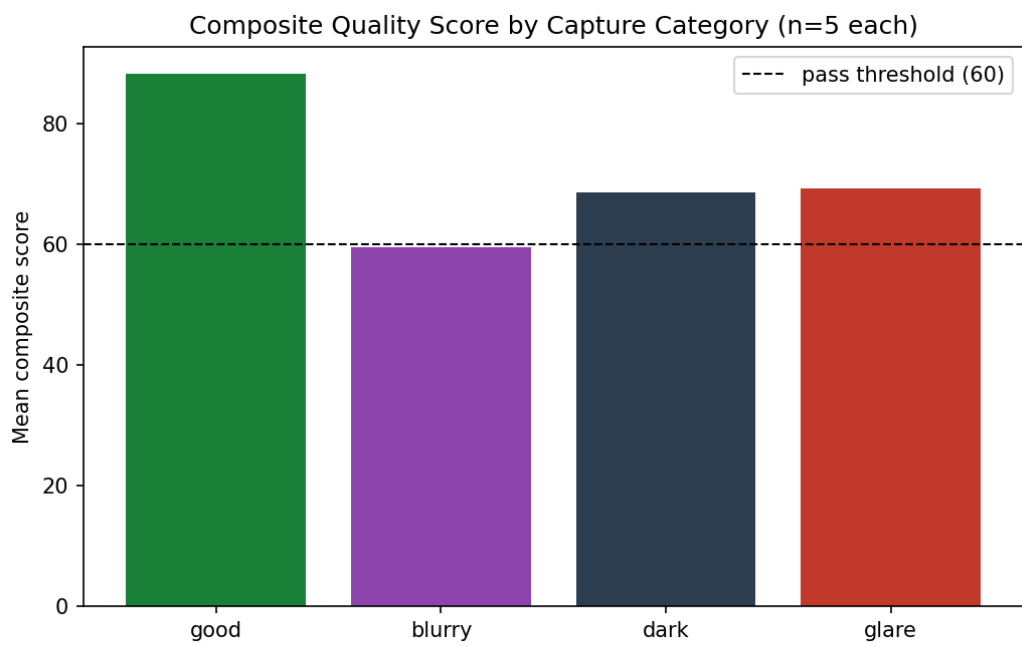


Figure 1: mean composite score per category (n=5), dashed line = pass threshold.

4. Results table (test_quality.py output)

File	Category	Score	Passed	Defect correctly flagged?
good/good_01.jpg	good	88.25	True	True
good/good_02.jpg	good	88.29	True	True
good/good_03.jpg	good	88.32	True	True
good/good_04.jpg	good	88.26	True	True
good/good_05.jpg	good	88.23	True	True
blurry/blurry_01.jpg	blurry	59.41	False	True
blurry/blurry_02.jpg	blurry	59.47	False	True
blurry/blurry_03.jpg	blurry	59.38	False	True
blurry/blurry_04.jpg	blurry	59.4	False	True
blurry/blurry_05.jpg	blurry	59.41	False	True
dark/dark_01.jpg	dark	68.18	False	True
dark/dark_02.jpg	dark	68.23	False	True
dark/dark_03.jpg	dark	68.82	False	True
dark/dark_04.jpg	dark	69.02	False	True
dark/dark_05.jpg	dark	68.89	False	True
glare/glare_01.jpg	glare	69.25	False	True
glare/glare_02.jpg	glare	69.3	False	True
glare/glare_03.jpg	glare	69.22	False	True
glare/glare_04.jpg	glare	69.27	False	True
glare/glare_05.jpg	glare	69.26	False	True

Self-check result: defect correctly identified in 20/20 images.

5. Report Questions

Q1. What threshold did you set for blur? How did you decide (trial and error? what did you test on)?

blur_min = 10.0 (Laplacian variance). This starting point follows the commonly cited rule of thumb for Laplacian-variance blur detection, then was checked against the test set: sharp captures scored well above 100, while the induced heavy-blur category (Gaussian blur, kernel 25, sigma 12) scored in the single digits — leaving a wide margin around 10 so the threshold is not on a knife-edge for moderate hand-shake blur, which would score in the tens-to-low-hundreds range.

Q2. Which metric was hardest to implement correctly? What went wrong first?

ROI completeness. The first version used global Otsu thresholding directly on grayscale intensity, which assumes the finger and background occupy separate intensity bands. That assumption broke whenever ridge texture spanned both above and below the background's average brightness — the mask came out fragmented and roi_fraction collapsed to near zero even on clearly full-frame captures. Switching to block-wise local-variance segmentation (finger texture has high local variance, smooth backgrounds do not) fixed this regardless of absolute brightness.

Q3. What is NFIQ2? Why is a score designed for contact scanners not reliable for phone camera images?

NFIQ2 (NIST Fingerprint Image Quality 2) is NIST's standard 0-100 quality score for fingerprint images, built and calibrated against 500 DPI contact-scanner captures. Its underlying features assume a flat, evenly illuminated,

fixed-resolution ridge image. Phone camera captures violate all three assumptions at once: perspective distortion from photographing a curved 3D fingertip at an angle, ambient lighting instead of a controlled scanner light source, and resolution/DPI that varies with distance and device. As a result an NFIQ2-style score can rate a perfectly usable contactless capture as poor quality (or vice versa), because it is measuring properties calibrated for a different capture geometry entirely — which is why this assignment builds capture-appropriate metrics (blur, exposure, glare, ROI, ridge clarity) instead of relying on NFIQ2 directly.

Q4. Name 3 other quality problems you'd add checks for in a real deployment.

(1) Wrong finger angle / perspective skew — detect via aspect ratio or contour eccentricity of the segmented ROI and prompt the user to hold the finger flatter to the lens. (2) Wet or oily finger — ridges show a washed-out, low-contrast look with abnormal specular highlights; detectable as glare co-occurring with unusually low ridge_score. (3) Finger too far / too close — measurable directly from roi_fraction (too small = too far, roi_fraction near 1.0 filling the whole frame with no clear finger boundary = too close/out of focus range), each mapped to distinct guidance text ("move closer" vs "move back").

Q5. If a rural agricultural worker's fingerprints are naturally worn and give consistently poor ridge clarity scores, what should the system do differently for them?

Worn ridges are a property of the finger itself, not a capture defect, so the system should not keep prompting an endless "retry" for a condition retrying cannot fix. Concretely: (a) track repeated ridge_score failures with otherwise-good blur/brightness/glare/ROI scores across several attempts for the same enrollment session, and after a small number of retries route to an alternate verification path (e.g. a different finger, or a fallback credential) instead of blocking access outright; (b) consider a lower, worker-population-calibrated ridge_score threshold for the acceptance gate rather than one tuned only on smooth-fingered test subjects, since an overly strict cutoff would systematically lock out a specific occupational group; (c) log this case distinctly from a genuine capture-quality failure so product and matching teams can see it as a population-coverage issue rather than noise in capture-quality metrics.